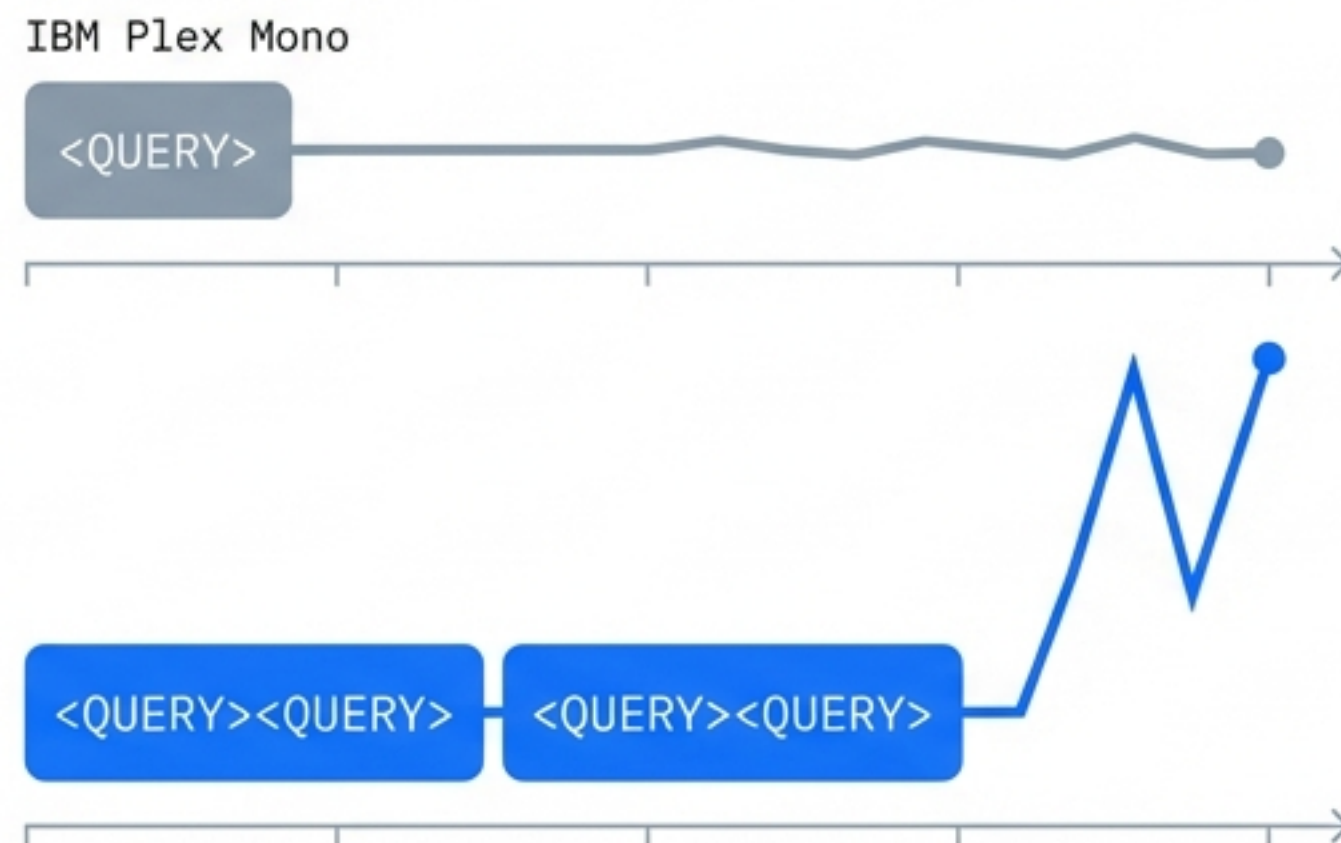


Prompt Repetition: A Simple Hack for Smarter LLMs (การทำซ้ำคำสั่ง: เทคนิคง่ายๆ เพื่อเพิ่มประสิทธิภาพ LLM)

Insights from Google Research on Non-Reasoning Models (ข้อมูลเชิงลึกจากงานวิจัย Google Research สำหรับโมเดลที่ไม่ใช้การให้เหตุผล)



Based on the paper 'Prompt Repetition Improves Non-Reasoning LLMs' by Leviathan et al. (2025).

The 'Free Lunch' Architecture (สถาปัตยกรรมแบบ "Free Lunch")

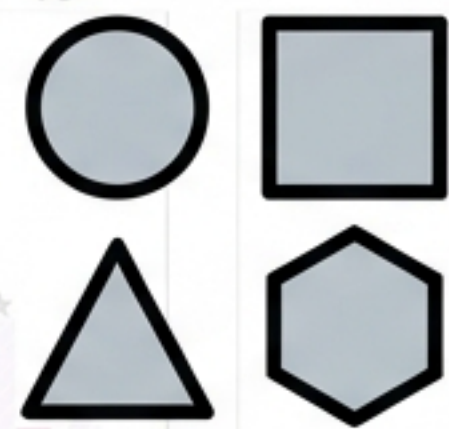
เพียงแค่ส่ง Prompt เดิมซ้ำสองครั้ง (Repeating the input)
สามารถเพิ่มความแม่นยำให้กับโมเดลมาตรฐานได้อย่างมหาศาล



High Success Rate: ชนะ 47
จาก 70 การทดสอบมาตรฐาน
โดยไม่แพ้เลยแม้แต่ครั้งเดียว
(0 Losses)



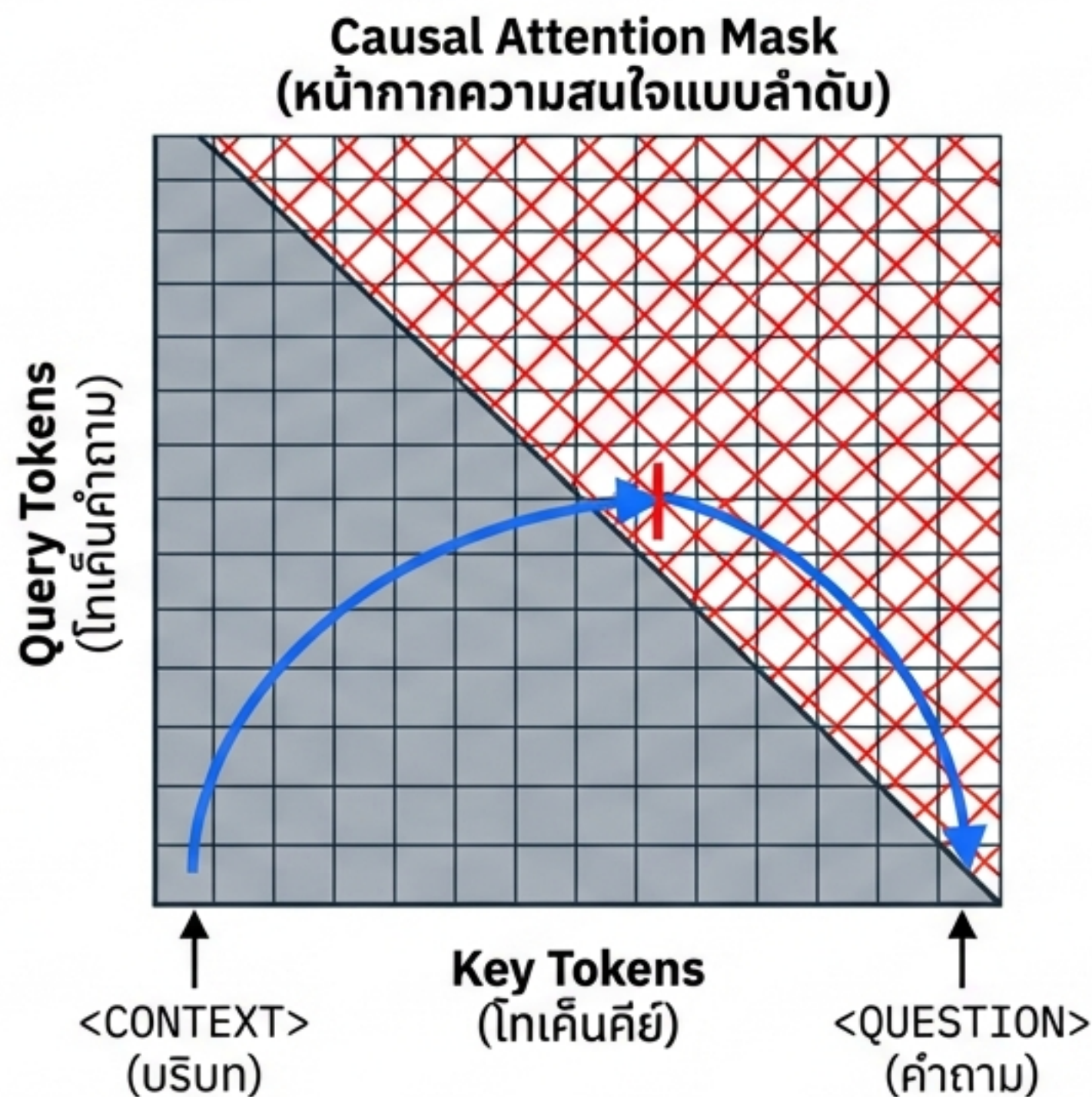
Zero Latency Cost: ไม่เพิ่ม
ไม่เพิ่มระยะเวลาในการตอบ
(Generation Latency) และ
ไม่เพิ่มจำนวน Token ที่ตอบ
กลับมา



Universal: ใช้ได้ผลกับทั้ง
GPT-4o, Gemini 2.0,
Claude 3.5/3.7, และ
DeepSeek V3

The Problem: Causal Attention 'Blind Spots'

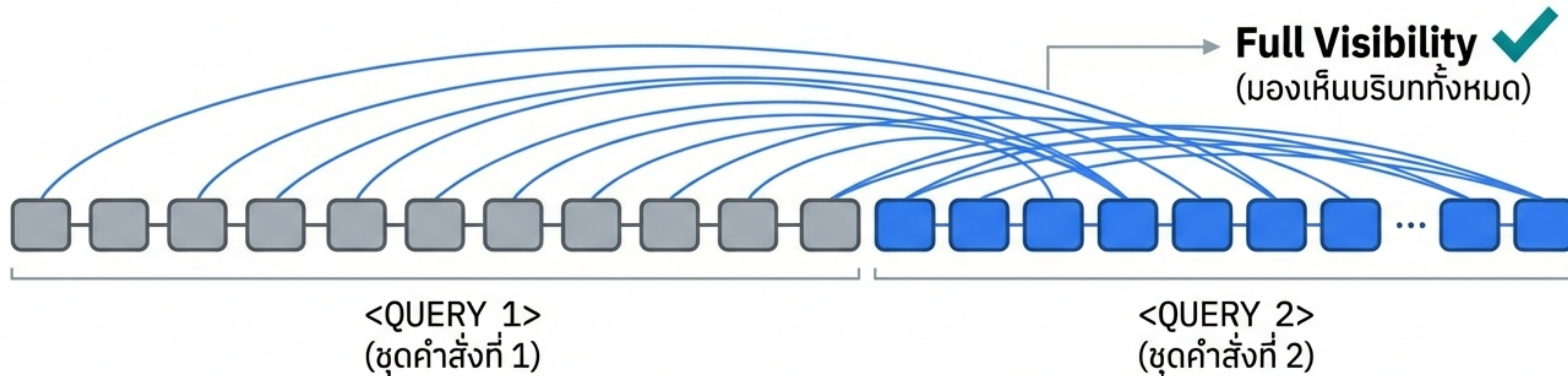
(ปัญหาของ Causal Attention: จุดบอดในการมองเห็นข้อมูล)



- **One-way Street:** โมเดล LLM อ่านข้อมูลจากซ้ายไปขวา (Left-to-Right)
- **The Disconnect:** หากเราใส่ข้อมูลบริบท (Context) ก่อนคำถาม โมเดลอาจประมวลผลบริบทโดยไม่รู้ว่าจะต้องหาคำตอบเรื่องอะไร ทำให้ความแม่นยำลดลงใน Prompt ที่ซับซ้อน

The Fix: Bidirectional-like Attention via Repetition

(ทางแก้: สร้างการมองเห็นทั่วถึงด้วยการทำซ้ำ)



เหมือนการอ่านประโยคซ้ำสองรอบ
เพื่อทำความเข้าใจบริบททั้งหมดก่อนเริ่มตอบคำถาม

IBM Plex Sans Thai/Mono: การทำซ้ำช่วยให้ Token ช่วงหลังสามารถ Attend (ให้ความสนใจ) กับ Prompt ชุดแรกได้ทั้งหมด แก้ปัญหาเรื่องลำดับข้อมูล (Prompt Ordering)

Validated Across the Industry

(ผ่านการทดสอบกับโมเดลชั้นนำทั่วโลก)

Google

Gemini 2.0 Flash, Gemini 2.0 Flash-Lite

OpenAI

GPT-4o, GPT-4o-mini

Anthropic

Claude 3 Haiku, Claude 3.7 Sonnet

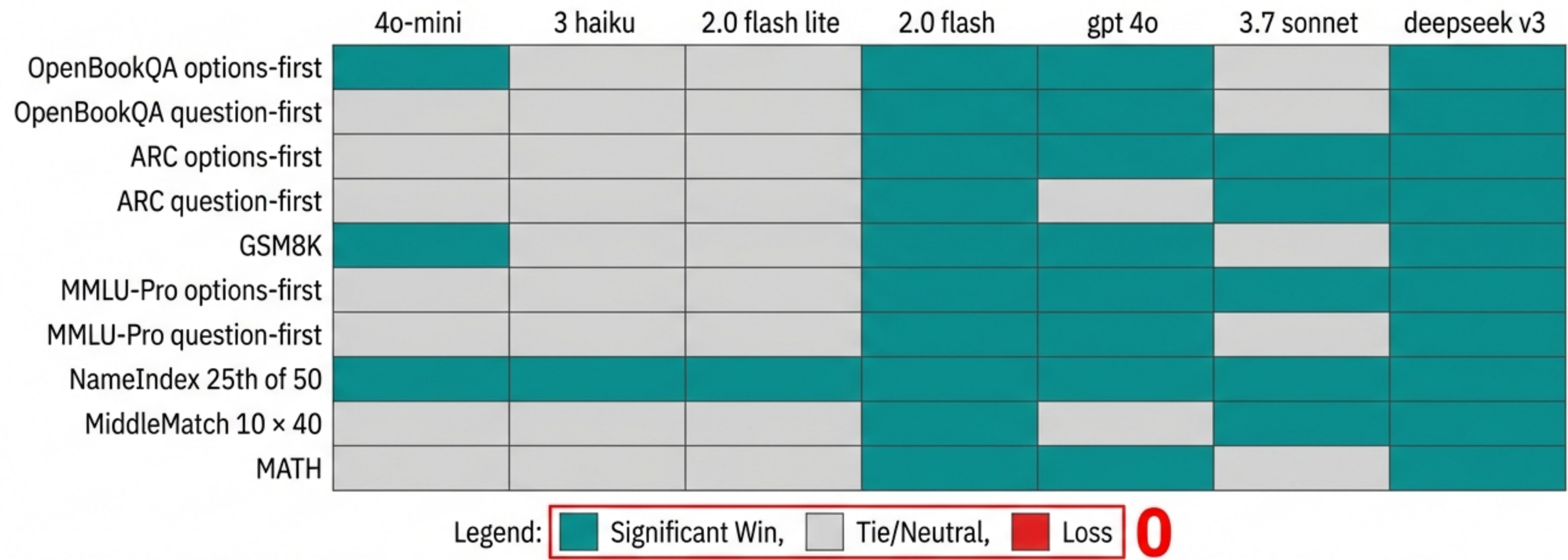
DeepSeek

DeepSeek V3

Benchmarks: MMLU-Pro, GSM8K, MATH, ARC, OpenBookQA, Custom Retrieval Tasks

การทดลองครอบคลุมโมเดลที่มีขนาดและความสามารถหลากหลาย เพื่อพิสูจน์ว่าเทคนิคนี้เป็น Universal Behavior ของ LLM

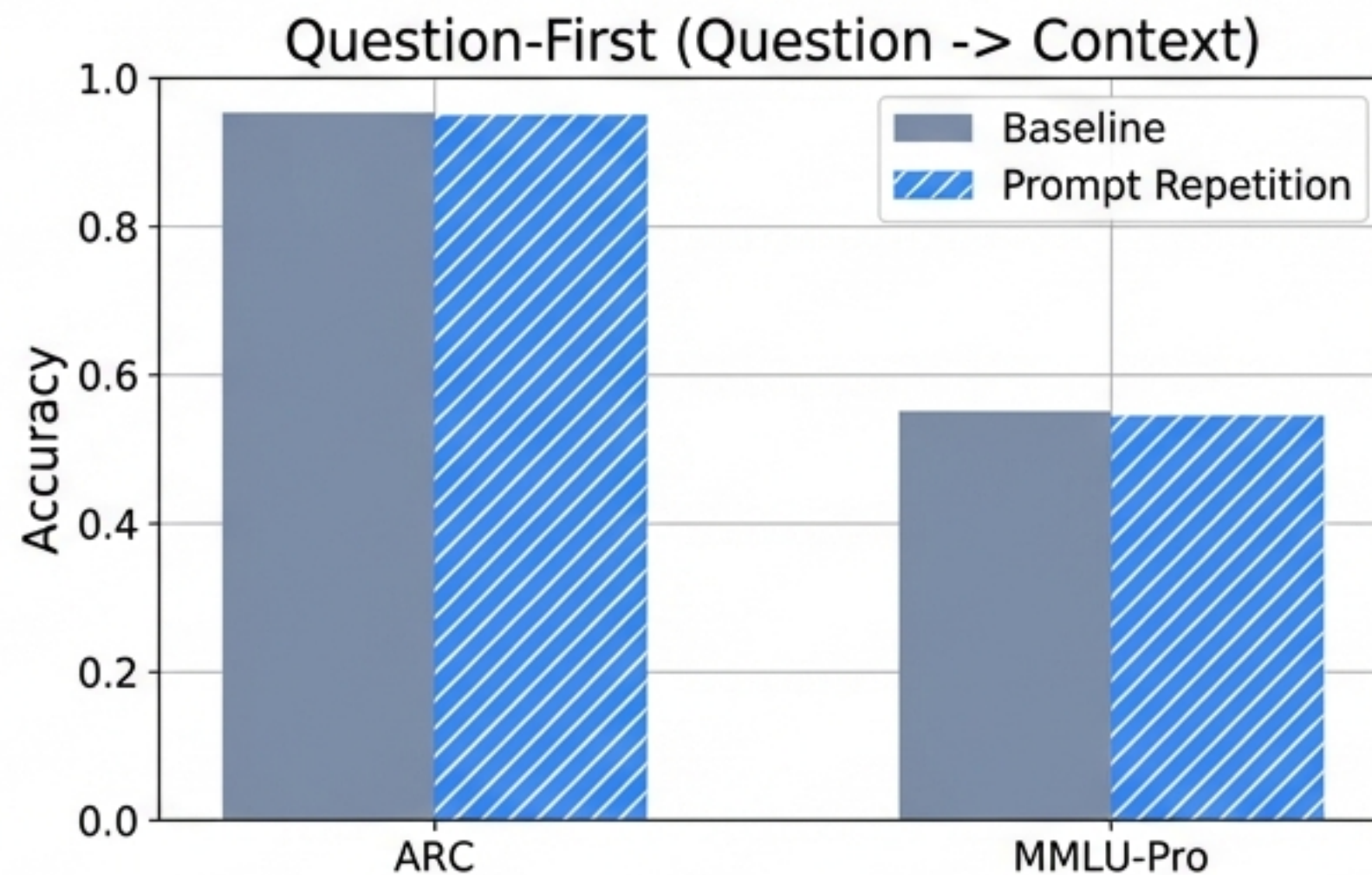
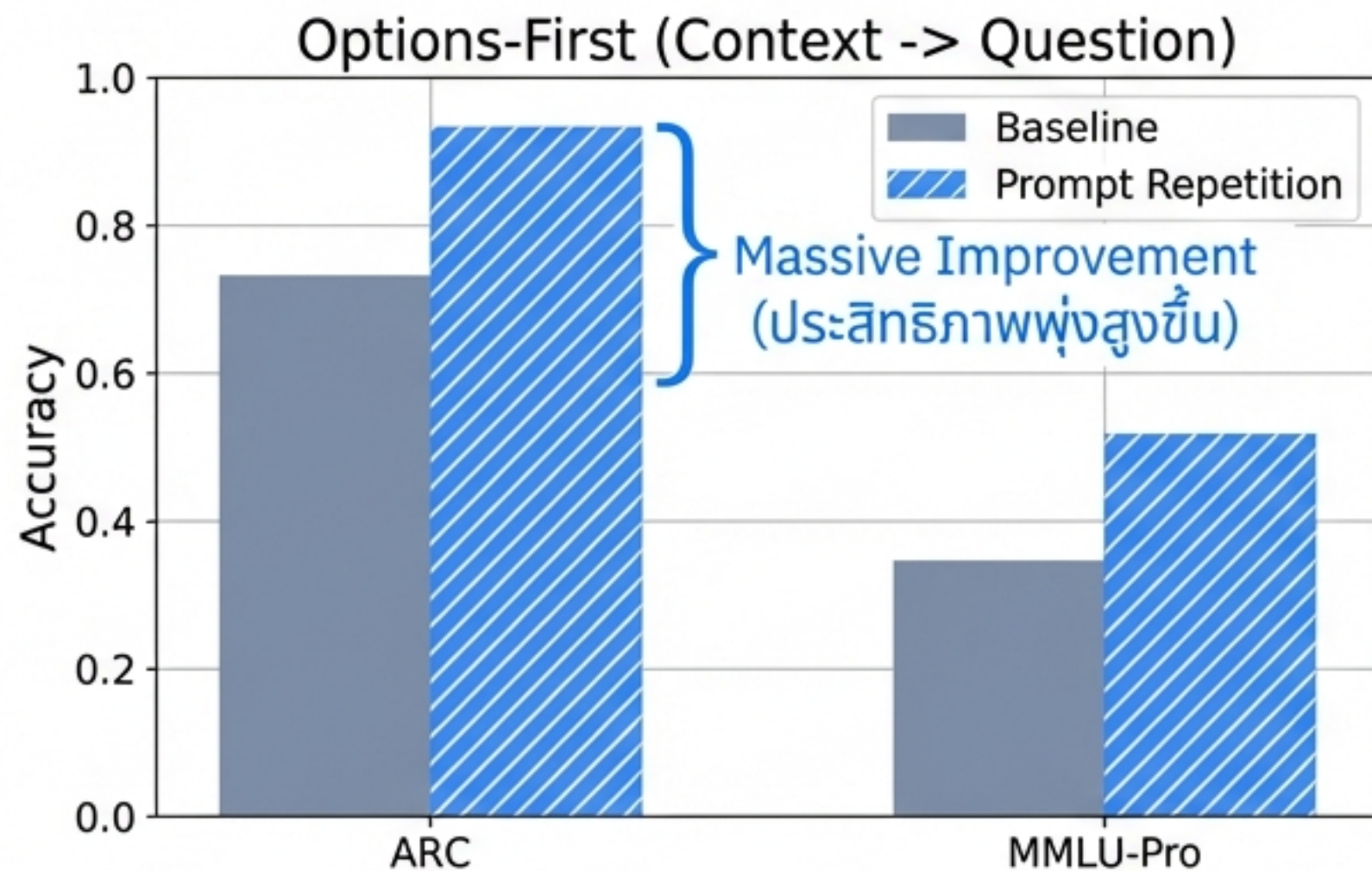
The Win Rate: 47 Wins, 23 Ties, 0 Losses (อัตราการชนะ: ชนะ 47 เสมอ 23 และไม่แพ้เลย)



Source: Heatmap Scorecard visualization based on the performance data comparing Prompt Repetition vs. Baseline across multiple benchmarks and models.

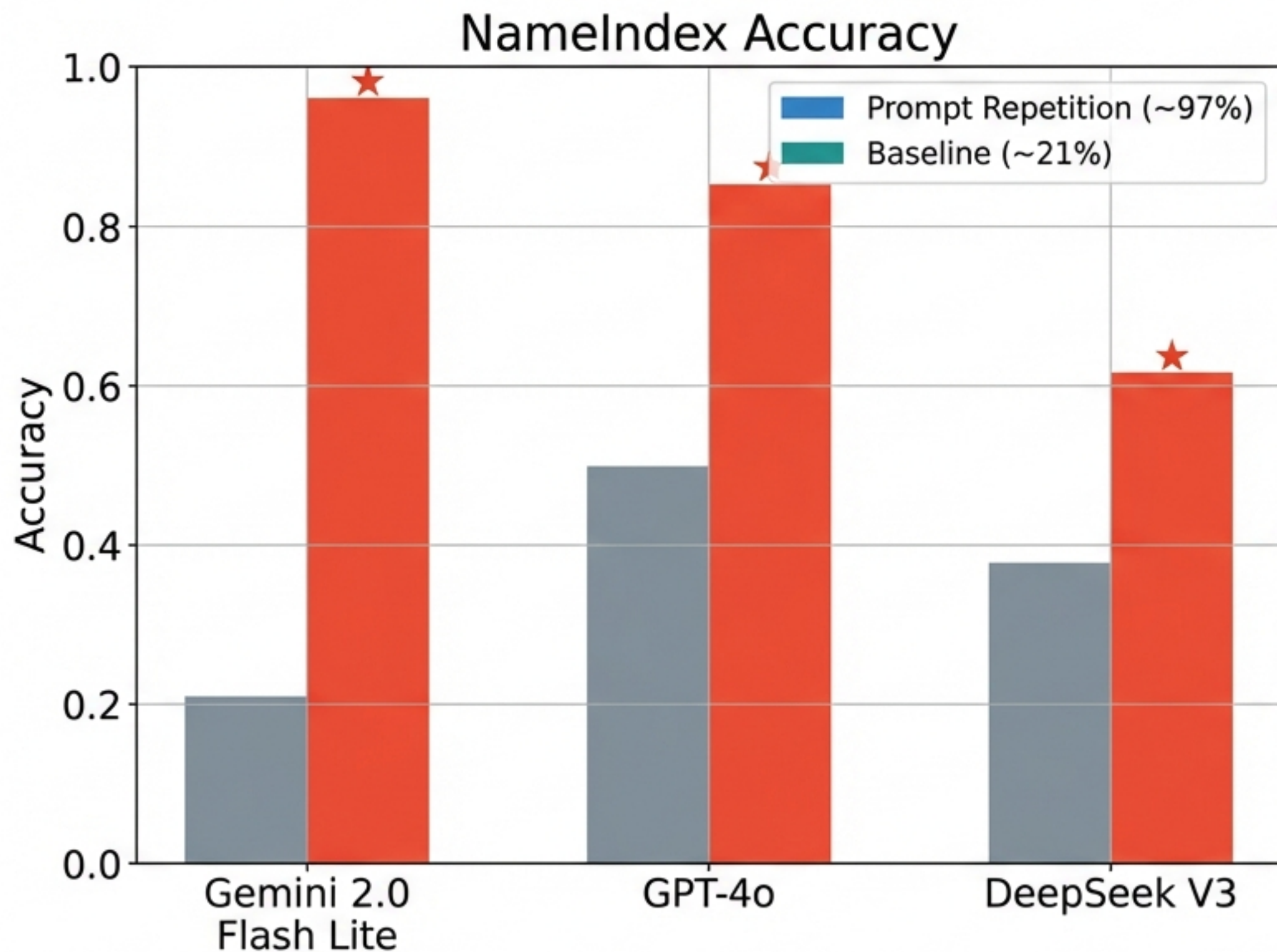
Key Takeaway: Prompt Repetition เป็นกลยุทธ์ที่มี Downside Risk ต่ำมาก คืออย่างน้อยที่สุดก็เสมอตัว แต่ส่วนมากจะช่วยให้ผลลัพธ์ดีขึ้น

Robustness Against Prompt Ordering (แก้ปัญหาลำดับการวาง Prompt)



- **Normal Behavior:** โมเดลมักทำได้แย่เมื่อเจอตัวเลือก (Options) ก่อนเจอคำถาม
- **With Repetition:** การทำซ้ำทำให้เราไม่ต้องกังวลเรื่องการจัดลำดับข้อมูล (Ordering) มากนัก โมเดลสามารถจัดการกับ Prompt ที่เขียนไม่เป็นระเบียบได้ดีขึ้น

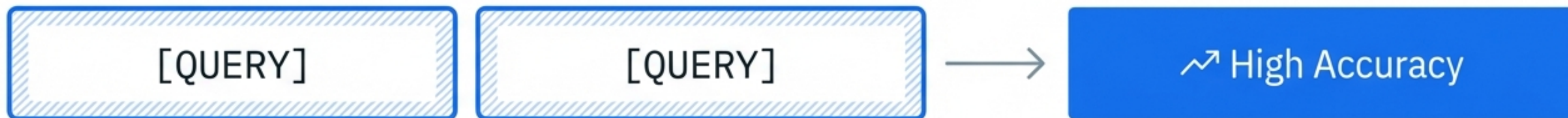
Supercharging Retrieval Tasks (เพิ่มพลังการค้นหาข้อมูลใน Context)



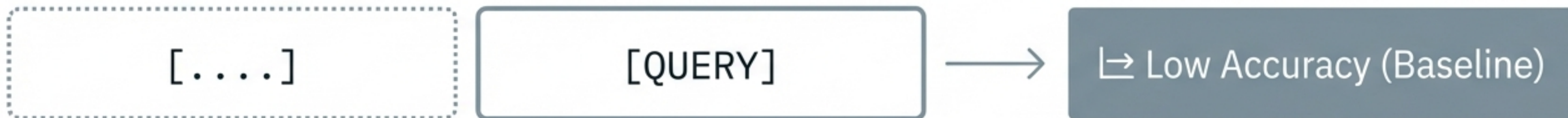
- **Task:** การค้นหาชื่อหรือข้อมูลเฉพาะเจาะจงในลิสต์ที่ยาวและซ้ำซ้อน (Retrieval within Context Window).
- **Result:** การทำซ้ำเปลี่ยนโมเดลรุ่นเล็ก (Lite/Flash) ให้มีความแม่นยำเทียบเท่าโมเดลรุ่นใหญ่ในงานประเภทค้นหาข้อมูล

It's Content, Not Just Length (สำคัญที่เนื้อหา ไม่ใช่แค่ความยาว)

Method A: Repetition



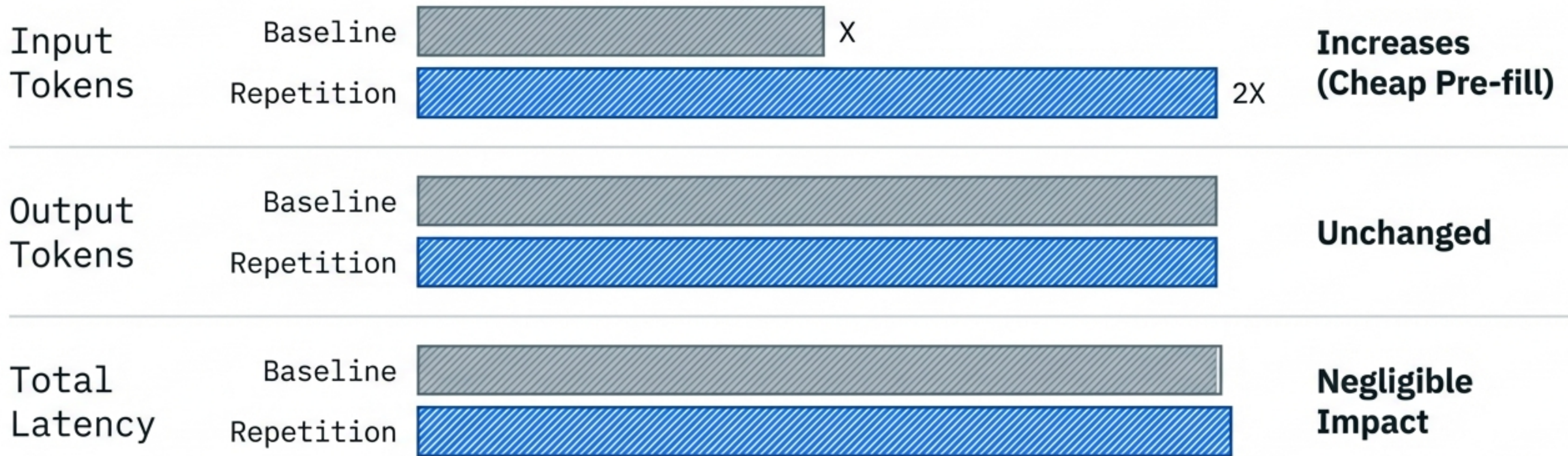
Method B: Padding



- ทิมวิจัยทดลองเติมจุด (Dots/Padding) ให้ Prompt ยาวเท่ากัน แต่ผลลัพธ์ **ไม่ดีขึ้น**
- Conclusion: โมเดลต้องการ 'ประมวลผลเนื้อหา' (Processing Content) ซ้ำอีกรอบ ไม่ใช่แค่ต้องการ Buffer พื้นที่ว่าง

Efficiency: The Mechanics of a ‘Free Lunch’

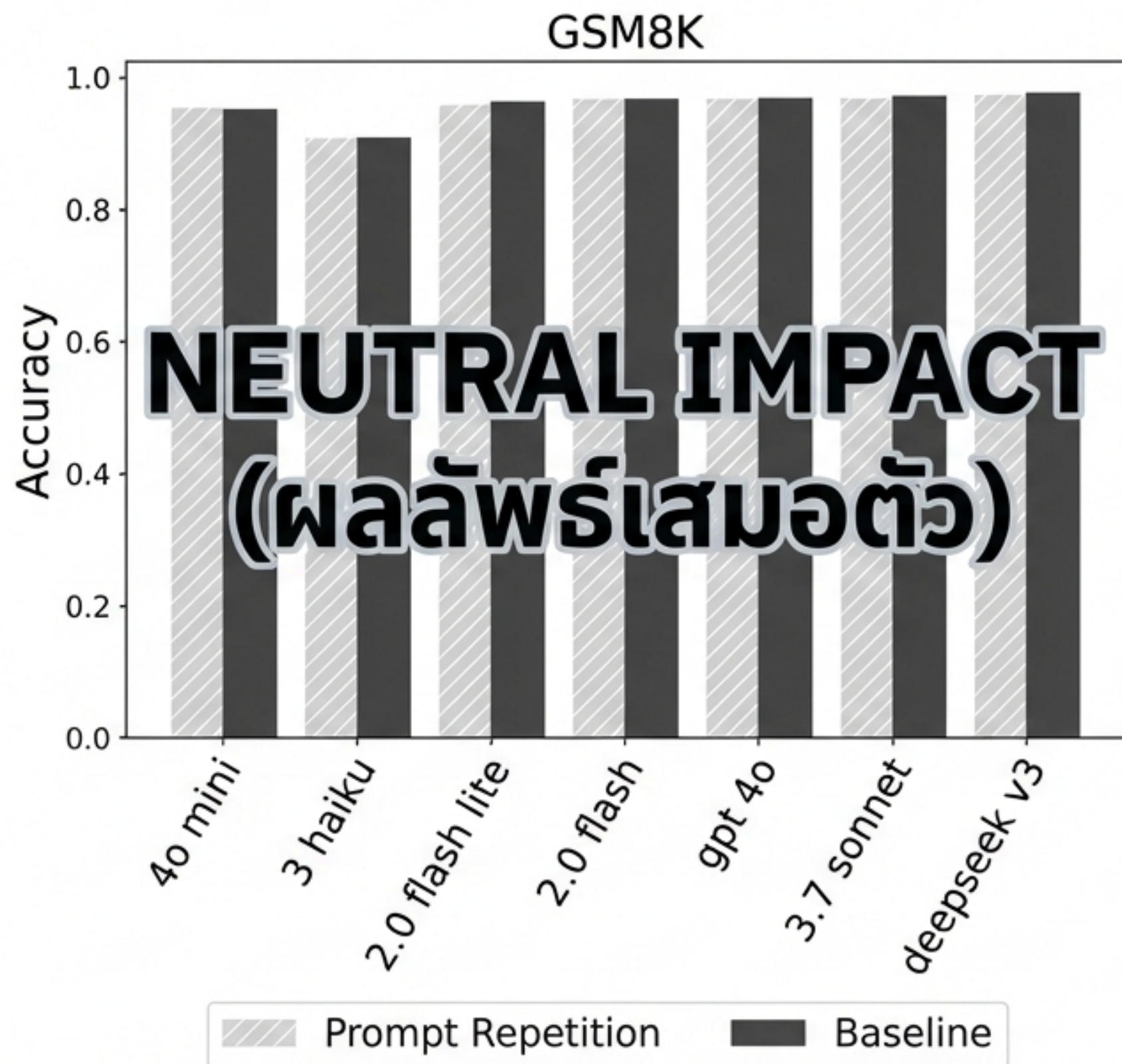
(ประสิทธิภาพ: ทำไมถึงเรียกว่าของฟรี?)



การซ้ำเกิดขึ้นในขั้นตอน **Pre-fill** ซึ่งทำงานแบบขนาน (Parallel) ได้ ทำให้ Time-to-First-Token แทบไม่เปลี่ยนแปลง เหมาะอย่างยิ่งสำหรับงานที่ต้องการ High Throughput โดยไม่ยอมเสียความแม่นยำ

The Exception: Reasoning Models & Chain of Thought

(ข้อยกเว้น: โมเดลสายคิดวิเคราะห์)



- เมื่อใช้คำสั่ง ‘Think step-by-step’ หรือใช้โมเดลสาย Reasoning (CoT) เทคนิคนี้จะให้ผลเสมอกัน (Neutral)
- **Why?:** โมเดลเหล่านี้มีกระบวนการ ‘ทวนซ้ำ’ ผ่านการสร้าง Chain of Thought ในตัวอยู่แล้ว
- **Recommendation:** ใช้ Prompt Repetition สำหรับโมเดลมาตรฐาน (Standard Models) ที่ต้องการคำตอบรวดเร็วและตรงไปตรงมา

Advanced Variants: Verbose & x3 (เทคนิคขั้นสูง: แบบเว้นวรรค และ แบบคูณสาม)

Deep Charcoal, IBM Plex Sans Thai Bold

Standard Repetition

Deep Charcoal, IBM Plex Sans Thai Bold)

[QUERY] + [QUERY] (IBM Plex Mono)

Verbose Repetition

Deep Charcoal, IBM Plex Sans Thai Bold)

[QUERY] + ["Let me repeat that:"] + [QUERY] (IBM Plex Mono)

x3 Repetition (Deep Charcoal, IBM & IBM Plex Mono)

[QUERY] + [QUERY] + [QUERY] ★ Best for Complex Retrieval (NameIndex)
(Sharp Teal, IBM Plex Sans Thai Regular)

Data Insight: ในงานที่ซับซ้อนมาก (เช่น NameIndex) การทำซ้ำ 3 รอบ (x3) ให้ผลลัพธ์ที่ดีกว่า 2 รอบอย่างมีนัยสำคัญ ส่วนการใช้ประโยคเชื่อม (Verbose) ช่วยแยกบริบทให้ชัดเจนขึ้นในบางโมเดล (Deep Charcoal, IBM Plex Sans Thai Regular)

Implementation Guide (คู่มือการใช้งานจริง)

When to use?

- ✓ ใช้กับ Standard / Fast models (GPT-4o mini, Haiku, Flash)
- ✓ ใช้กับงาน Retrieval, QA, หรือการทำตาม Format ที่เคร่งครัด
- ✗ ไม่จำเป็นต้องใช้กับ Reasoning/O1-like models หรือเมื่อใช้ CoT

Python Snippet

```
# Simple Implementation Strategy
user_prompt = "Find the anomaly in this log..."

messages = [
    {
        "role": "user",
        "content": f"{user_prompt}\n\n{user_prompt}"
    }
]
```

A simple repetition moves the computation from generation (expensive) to **pre-fill** (cheap/parallel).